

O analiză comparativă a familiei scheduling din SAT Competition 2024

Berindeie Adrian, Buzuriu Roxana, Constantinescu Mihai, și Felea Irina

Universitatea de Vest din Timișoara
<https://www.uvt.ro>
Inginerie Software

Rezumat Această lucrare se concentrează pe optimizarea solverului MiniSat pentru benchmark-uri din competiția SAT 2024, concentrându-se pe algoritmul de învățare a clauzelor determinate de conflict (CDCL) și euristica de ordonare a variabilelor. Rezultatele preliminare indică câștiguri de performanță pentru diferite configurații de benchmark-uri, testate pe trei sisteme de operare diferite, Windows, Linux și MacOS. Cele mai bune rezultate au fost obținute pe MacOS, urmate de Linux, în timp ce cele mai slabe performanțe au fost observate pe Windows. Rezultatele obținute au fost determinate și de numărul de variabile și numărul de clauze, dar cel mai important aspect este CDCL și cât de rapid găsește conflictele pentru a le elimina. În plus, sunt documentate practicile de inginerie software utilizate, cum ar fi diagramele UML și documentația cuprinzătoare a codului, pentru a facilita dezvoltarea și întreținerea viitoare.

Keywords: SAT, Satisfiabilitate, MiniSat, CDCL, DPLL, Solvere, Euristici, SAT Competiton

1 Introducere

Problemele de satisfiabilitate booleană (SAT) reprezintă o componentă esențială a logicii computaționale, având aplicații extinse în domenii precum verificarea formală a sistemelor software și hardware, planificarea automată și optimizarea [2]. Solverele de tip SAT, cum ar fi MiniSat [6], sunt utilizate pentru a determina dacă o formulă booleană poate fi satisfăcută. Se poate confirma că este satisfăcută atunci când există o soluție, adică dacă s-a găsit o atribuire de valori de adevăr pentru variabile care să facă întreaga formulă adevărată. Un SAT solver este un instrument care determină dacă o formulă propozițională are o atribuire care o face adevărată. Performanța acestor solvere a cunoscut îmbunătățiri semnificative în ultimele decenii, permițând rezolvarea unor formule cu milioane de variabile și clauze [9].

MiniSat, este un solver SAT care se bazează pe algoritmul CDCL (Conflict-Driven Clause Learning)[15,11], fiind bazat pe învățarea de conflicte și reordonarea dinamică a variabilelor [7]. Acesta s-a remarcat prin eficiență și simplitate, astfel putem spune că a devenit un reper în dezvoltarea de soluții avansate

pentru SAT [7]. Algoritmul CDCL folosit în MiniSat a adus îmbunătățiri semnificative față de DPLL (Davis-Putnam-Logemann-Loveland) [3,4], introducând mecanisme precum generarea clauzelor de conflict, deciziile euristice pentru selectarea variabilelor și reluarea adaptivă a căutării prin backtracking cronologic. Aceste îmbunătățiri au condus la o creștere semnificativă a eficienței în rezolvarea instanțelor complexe de SAT.

De-a lungul timpului, solvele SAT bazate pe CDCL, precum MiniSat, au fost optimizate prin integrarea unor tehnici suplimentare, cum ar fi reordonarea dinamică a clauzelor și metoda de propagare utilizând doi literalii urmăriți [8]. Aceste îmbunătățiri au avut un impact semnificativ asupra eficienței solvelelor. Reordonarea dinamică a clauzelor se referă la ajustarea continuă a ordinii în care clauzele sunt evaluate sau stocate în timpul execuției solverului. Scopul acestei tehnici este de a prioritiza clauzele relevante, adică acelea care sunt mai susceptibile să conducă la conflicte sau care influențează deciziile curente ale solverului. Prin utilizarea unor criterii euristice, cum ar fi frecvența accesării clauzelor sau poziția acestora în analiza conflictelor, solverul poate accelera procesul de deducție, reducând semnificativ timpul necesar pentru a găsi o soluție sau pentru a demonstra insatisfiabilitatea unei formule.

Pe de altă parte, propagarea folosind doi literalii urmăriți (în engleză, *two-watched literals*) este o tehnică care optimizează procesul de *propagare unitară*. Aceasta constă în deducerea valorilor variabilelor necesare pentru ca o clauză să fie satisfăcută, având în vedere atribuțiile curente. În loc să verifice toți literalii dintr-o clauză la fiecare pas, solverul urmărește doar doi literalii neîndepliniți din fiecare clauză. Atunci când unul dintre acești literalii devine fals, solverul îi verifică pe restul, pentru a găsi un înlocuitor neatribuit sau adevărat, menținând astfel clauza satisfăcută. Dacă nu se găsește un astfel de literal și celălalt literal urmărit este, de asemenea, fals, clauza devine conflictuală, declanșând propagarea sau analiza conflictului [14]. Această abordare reduce semnificativ numărul de verificări necesare, îmbunătățind astfel performanța generală a solverului. Prin integrarea acestor tehnici, solvele CDCL au devenit capabile să abordeze probleme complexe de satisfiabilitate booleană, menținând o eficiență ridicată în procesele de căutare și deducție, oferind astfel o bază solidă pentru dezvoltarea continuă a acestui domeniu.

Competițiile SAT oferă o platformă de testare riguroasă pentru evaluarea și compararea noilor soluții în rezolvarea problemelor SAT. Aceste evenimente adună dezvoltatori pentru a prezenta și testa cele mai recente soluții, contribuind astfel la progresul constant al acestui domeniu și evidențiind rolul esențial al implementărilor optimizate și a algoritmilor eficienți [10]. MiniSat și variațiile sale au fost supuse unor îmbunătățiri constante în cadrul acestor competiții, care includ strategii avansate de propagare și gestionare a clauzelor învățate. Aceste tehnici s-au dovedit esențiale pentru reducerea timpului de rulare și creșterea performanței generale a solverului.

În acest context, proiectul nostru își propune să implementeze și să evalueze modificări specifice în MiniSat, axate pe optimizarea algoritmilor, cu scopul de a îmbunătăți eficiența în soluționarea problemelor.

Modificările propuse vor fi testate pe un set de benchmark-uri din familia scheduling, utilizate în competiția SAT 2024 [16], permițând o analiză detaliată a impactului asupra performanței și o comparație cu versiunea inițială a MiniSat.

În contextul solverelor SAT, un benchmark reprezintă un set de probleme standardizate utilizate pentru a evalua și compara performanța diferitelor solvere. Aceste benchmark-uri sunt esențiale pentru a asigura o evaluare obiectivă și pentru a identifica punctele forte și slabe ale fiecărui solver.

Benchmark-urile utilizate în competițiile SAT sunt de obicei împărțite în familii, fiecare reprezentând un tip specific de problemă care este sau nu rezolvată. De exemplu, familia "scheduling" cuprinde probleme legate de programarea activităților sau planificarea spre exemplu a unui orar școlar, în timp ce alte familii pot include probleme care implică analiza circuitelor pentru a identifica posibile defecte, planificări automate sau care conțin formule generate aleatoriu pentru a testa performanța generală a solverelor. Această clasificare permite o evaluare mai detaliată a performanței solverelor în funcție de tipul de problemă abordată.

Noi am ales să utilizăm benchmark-uri din familia *scheduling*, alegerea fiind motivată de aplicabilitatea practică a acestor probleme, care sunt frecvent întâlnite în industrie și cercetare. Acestea includ domenii precum planificarea producției, a orarului sau alocarea resurselor și programarea activităților [13]. De exemplu, problemele de tip *scheduling* sunt esențiale în optimizarea proceselor de producție, unde se urmărește maximizarea eficienței resurselor și minimizarea timpilor de execuție.

În proiectul nostru vom folosi MiniSat *simp* pentru rularea fișierelor, acesta fiind o variantă îmbunătățită și simplificată a MiniSat-ului core [5]. Pentru un studiu cât mai elaboros, vom folosi trei sisteme de operare diferite, Windows, Linux și MacOS. Prin această cercetare vom evidenția atât dificultățile întâmpinate, cât și posibilele direcții de optimizare viitoare pentru solvele de tip SAT.

2 Descrierea Problemei

În cadrul acestui studiu, ne propunem să evaluăm performanța solverului SAT, MiniSat, utilizând benchmark-uri din familia "scheduling". Analiza va fi realizată comparativ pe trei sisteme de operare distincte: Windows, Linux și MacOS. Această abordare ne va permite să identificăm eventualele diferențe de performanță și să determinăm influența specifică a fiecărui sistem de operare asupra eficienței solverului.

Motivul acestui studiu este bazat pe faptul că, în literatura de specialitate, nu există lucrări clare care să compare în mod direct performanța solverelor SAT între diferite sisteme de operare. Cu toate acestea, în diverse discuții neoficiale și forumuri tehnice, s-a susținut că ar putea exista diferențe semnificative între platforme, cauzate de modul în care fiecare sistem gestionează memoria, resursele hardware sau compilatorii specifici. Prin urmare, considerăm esențială realizarea unei analize riguroase pentru a confirma sau infirma aceste ipoteze și pentru a

oferi o bază solidă în luarea deciziilor cu privire la platforma optimă pentru rularea solverelor SAT. Prin această comparație, vom obține o înțelegere mai profundă a comportamentului MiniSat în medii de operare diferite și vom putea formula recomandări practice pentru utilizarea optimă a acestuia, în funcție de platforma aleasă

2.1 Detalii Tehnice

Pentru abordarea problemei, am structurat analiza astfel încât să includă următoarele etape:

- **Setarea mediului de testare:** Am instalat MiniSat pe cele trei sisteme de operare și am configurat mediul astfel încât să eliminăm eventualele variații externe. Am utilizat aceeași versiune de MiniSat pe Linux și MacOS, precum și aceleași benchmark-uri din familia "scheduling".
- **Optimizarea configurației MiniSat:** Pentru a evalua performanța în cele mai bune condiții, am realizat ajustări ale parametrilor solverului, precum euristicele de selecție a variabilelor.
- **Execuția benchmark-urilor:** Am rulat fiecare benchmark în mod repetat pe cele trei sisteme de operare pentru a colecta date precise privind timpul de rulare, utilizarea memoriei și numărul de conflicte rezolvate.
- **Soluționarea tehnică a diferențelor de performanță:** Pe baza rezultatelor obținute, am investigat cauzele diferențelor de performanță observate. Acestea pot fi legate de arhitectura hardware a sistemului, de modul în care fiecare sistem de operare gestionează memoria sau de performanța diferită a compilatorului.

2.2 Soluționarea Problemei

Abordarea tehnică a problemei a constat în evaluarea fiecărui pas din procesul de rulare a solverului pe cele trei sisteme de operare, identificarea cauzelor posibile ale variațiilor și implementarea unor soluții pentru uniformizarea performanțelor. Am efectuat comparații directe asupra consumului de resurse, identificând impactul fiecărei componente (CPU, memorie) asupra performanței globale.

Rezultatele experimentelor ne-au oferit informații detaliate despre modul în care fiecare sistem gestionează rularea benchmark-urilor și ne-au ghidat în ajustarea parametrilor MiniSat pentru a obține o performanță consistentă și optimizată.

3 MiniSat

MiniSat este unul dintre cele mai populare și eficiente solvere SAT open-source, utilizat pe scară largă în cercetare și industrie. Dezvoltat pentru a rezolva probleme complexe de satisfiabilitate booleană, MiniSat a devenit un reper datorită designului său modular, eficienței ridicate și posibilităților de personalizare. Versiunile sale, inclusiv MiniSat standard și MiniSat simp, sunt frecvent utilizate

în competițiile SAT și reprezintă puncte de plecare pentru dezvoltarea de noi solvere.

3.1 Diferențe între MiniSat standard și MiniSat simp

MiniSat este un solver SAT open-source, apreciat pentru eficiența și flexibilitatea sa. Versiunea standard a MiniSat implementează algoritmul *Conflict-Driven Clause Learning* (CDCL), acesta utilizând tehnici esențiale precum propagarea unitate și învățarea din conflicte. Pe de altă parte, MiniSat simp extinde funcționalitățile versiunii standard prin integrarea unor tehnici avansate de simplificare a formulelor SAT, precum eliminarea clauzelor redundante. Aceste tehnici reduc dimensiunea formulei și contribuie la îmbunătățirea performanței solverului, fiind utile în cazul problemelor complexe. MiniSat simp oferă avantaje clare, fiind utilizat frecvent în competițiile SAT și în cercetările care implică scenarii complexe. [17]

3.2 Instalare MiniSat

Instalare MiniSat pe Linux Pentru a instala MiniSat pe sistemul de operare Linux, este nevoie de clonarea proiectului de repository-ul de GitHub, folosind comanda prezentată în Secvența de cod 1.1. După clonarea proiectului, din directoriul MiniSat-ului, se rulează comanda prezentată în Secvența de cod 1.2 pentru a rula codul într-un mod mai permisiv. Această comandă setează un flag specific pentru compilatorul de C++, transformând erorile mai puțin critice în avertismente și astfel ajutând la compilarea codului. Comanda prezentată în Secvența de cod 1.3 este o comandă necesară pentru cazul în care sistemul nu are deja comanda "make", comandă necesară pentru a crea un executabil din codul C++. După rularea comenzilor se navighează în locația executabilului prin comanda prezentată în Secvența de cod 1.4, iar din acest directoriu putem folosi programul MiniSat.

Secvența de cod 1.1: Clonare proiect

```
git clone https://github.com/irinafelea/minisat.git
```

Secvența de cod 1.2: Compilare permisivă

```
cd minisat/
make CXXFLAGS="-fpermissive"
```

Secvența de cod 1.3: Instalarea comenzii make

```
sudo apt install make
```

Secvența de cod 1.4: Navigarea către directoriul executabilului

```
cd build/release/bin
```

Instalare MiniSat pe Windows Instalarea MiniSat pe sistemul de operare Windows este precedată de instalarea unor programe și configurarea căilor de acces. Pentru început, este necesară instalarea subsistemului Unix pentru Windows, Cygwin¹, rularea fișierului `setup.exe`, în urma căruia Cygwin va crea directoarele în directoriul `../cygwin/`. Apoi este necesară descărcarea software-ului CMake² și extragerea fișierelor într-un director cu denumirea `C:\CMake`. După realizarea instalărilor necesare, se configurează căile de acces în PATH. Pentru a configura căile de acces este necesară deschiderea setărilor sistemului, iar în "**Environment Variables**", la variabila `Path` sunt adăugate căi de acces către `MinGW\bin` și `CMake\bin`. Pentru a instala Minisat, acesta trebuie descărcat de pe pagina oficială a programului³. Se deschide subsistemul Cygwin și se navigează în directoriul Minisat. Din acest directoriu se rulează comanda prezentată în Secvența de cod 1.5, comandă care specifică generatorul de fișiere Makefiles pentru Minimalist GNU for Windows (MinGW) și creează un fișier Makefile compatibil cu `mingw32-make`. În Secvența de cod 1.6 este prezentată comanda care compilează fișierele generate anterior și creează executabilul `minisat.exe`.

Secvența de cod 1.5: Generare fișiere Makefile

```
cmake -G "MinGW Makefiles" ...
```

Secvența de cod 1.6: Generare fișiere Makefile

```
cmake --build
```

3.3 Detalii de implementare

Structuri de date folosite În această secțiune vom prezenta principalele structuri de date utilizate în cadrul aplicației MiniSat. Definirea acestor structuri de date a fost realizată în cadrul fișierului `minisat/mtl/SolverTypes.h`.

struct Lit - Reprezentarea unui literal logic utilizând o variabilă de tip `int`, astfel făcând posibilă și stocarea semnului literalului.

class lbool - Reprezentarea unei valori logice. Această clasă este creată pentru a optimiza comparația între o variabilă și o constantă, precum și pentru a se asigura că `gcc` realizează suficientă propagare a constantelor. Aceste aspecte sunt precizate pe linia 89. Această clasă mai conține în plus suprascrierea unor operatori pentru a ușura lucrul cu acest tip de variabilă în contextul rezolvării unei probleme de satisfiabilitate în logica computațională, cum ar fi: `==`, `!=`, `^`, `&&`, `||`. Aceste variabile au fost create cu scopul de a reprezenta 3 valori, chiar dacă acestea sunt definite de tipul `uint8_t`: `true`, `false`, `undefined`. Astfel, pe liniile 123–131, putem observa definirea următoarelor constante:

¹ Cygwin <https://www.cygwin.com> [accesat în 02.11.2024]

² CMake <https://cmake.org/download/> [accesat în 03.11.2024]

³ The MiniSat Page <http://minisat.se/MiniSat.html> [accesat în 05.11.2024]

- `l_True` (`lbool((uint8_t)0)`)
- `l_False` (`lbool((uint8_t)1)`)
- `l_Undef` (`lbool((uint8_t)2)`)

class Clause - Reprezentarea unei clauze logice. Această clasă reține informațiile de bază precum lista de literali și lungimea acestora dar și alte informații mai avansate precum dacă această clauză a fost învățată deja (proprietatea `learned`). Totuși este precizat că nu se poate folosi în mod direct constructorul acestei clase, ci se va folosi clasa prieten `ClauseAllocator` care se ocupă de alocarea corectă și eficientă a memoriei pentru această structură de date.

Pașii parcurși de algoritm În Figura 1 este reprezentată diagrama de secvențe a programului MiniSAT. Analiza pașilor principali executați de programul MiniSat începe cu fișierul `minisat/core/Main.cc`. La începutul funcției `main()` putem observa cum programul analizează argumentele primite din linia de comandă și extrage calea către fișierul de intrare (dacă există) și opțiunile selectate cum ar fi: verbozitatea, limita de timp de execuție pe procesor permisă (în secunde), limita de memorie utilizată și alegerea de a valida sau nu antetul DIMACS în timpul parcurgerii (liniile 63-68).

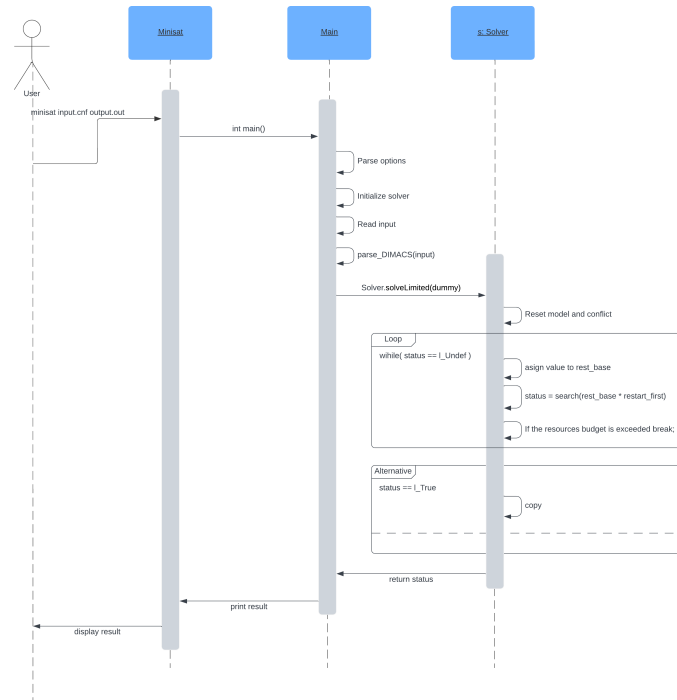


Figura 1: Diagrama de secvență a sistemului

După acest pas programul inițializează solver-ul S, extrage timpul inițial, limitează resursele bazat pe argumentele primite și citește datele de intrare de la tastatură sau dintr-un fișier de intrare după caz. Pe linia 95 programul începe să parcurgă datele de intrare și să adauge clauze în interiorul lui S în cadrul funcției `parse_DIMACS`. Se încearcă simplificarea clauzelor. Dacă acesta eșuează, se declară că formula este nesatisfiabilă iar execuția este oprită.

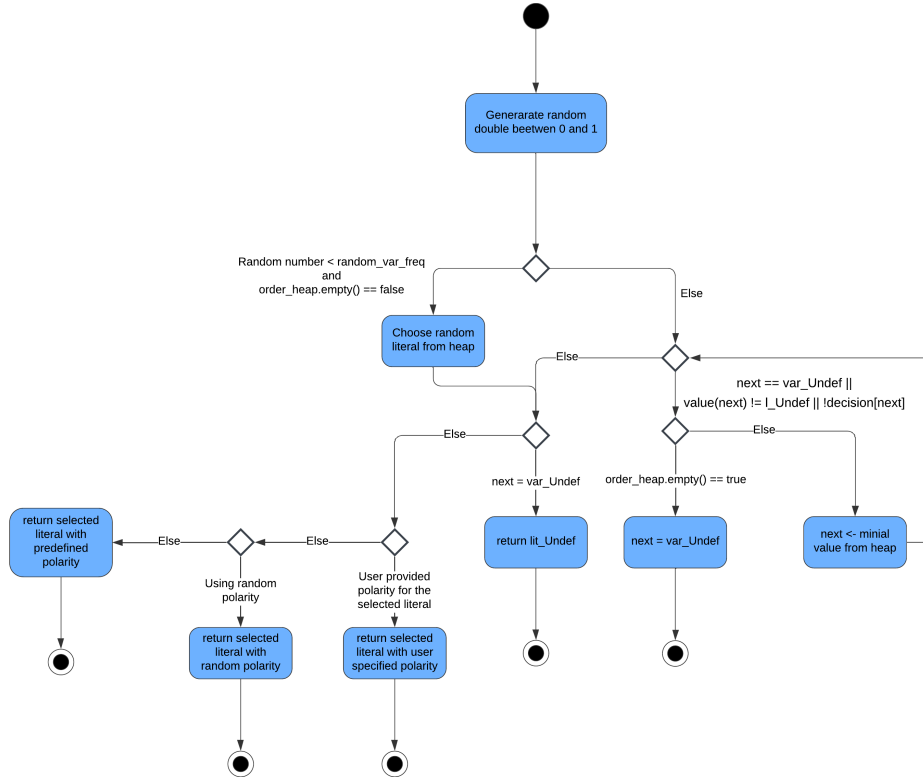
Rezolvarea propriu-zisă a problemei începe pe linia 124 prin apelarea metodei `S.solveLimited(dummy)`. Implementarea acestei metode se poate găsi în fișierul `minisat/core/Solver.cc`. Această metodă începe prin resetarea atributelor `model` și `conflict` urmată de verificarea dacă solver-ul este într-o stare validă. Căutarea unei soluții se realizează în interiorul buclei dintre liniile 865-870 care se execută atât timp cât valoarea variabilei `status` este `l_Undef`. Principala acțiune de căutare a unei soluții se realizează în cadrul metodei `search()` care va fi detaliată în Subsecțiunea 3.3. După găsirea unui status acesta este procesat și afișat utilizatorului alături de restul statisticilor adunate în timpul execuției.

Detalierea funcțiilor principale

Metoda `simplify()` - În implementarea curentă, această funcție doar elimină clauzele care sunt deja satisfăcute, reducând dimensiunea problemei, ajutând astfel la accelerarea procesului de căutare a unei soluții. Aceasta parcurge clauzele care sunt satisfăcute cu valorile deja atribuite unor literalii, până în acel moment al execuției.

Metoda `luby()` - Metoda luby este folosită pentru a calcula numărul de resetări permise în cadrul metodei `search`. Se alege să se folosească această metodă de calcularea a restricției dacă utilizatorul setează flag-ul `luby_restart` ca activ. În caz contrar restricția va fi calculată pe baza unei creșteri exponențiale a valori precedente permise. Pasând către aceasta incrementul de resetare și valoarea curentă a restricției se crează o secvență de numere în care există o creștere exponențială. Această secvență Luby are forma 1 1 2 1 1 2 4 1 1 2 1 1 2 4 8

Metoda `pickBranchLit()` - Metoda selectează următoarea decizie, atât literalul cât și polaritatea acestuia. Se alege o valoare aleatoare întreagă între 0 și 1. Dacă acesta este mai mică decât variabila `random_var_freq` și heap-ul în care se rețin variabilele neasignate nu este gol, atunci se alege o variabilă la întâmplare din heap.

Figura 2: Diagrama de activitate a metodei *pickBranch*

Dacă această condiție nu este adevărată cât timp variabila **next** este egală cu **var_Undef** sau valoarea pentru **next** este diferită de **l_Undef** sau **decision[next]** încă este fals se verifică heap-ul. Dacă acesta este gol, lui **next** i se asignează valoarea **var_Undef** și se iese din buclă. Dacă nu, lui **next** i se dă valoarea minimă din heap.

Pentru a se alege polaritatea, prima dată se verifică dacă **next** este **var_Undef**. În acest caz se returnează **lit_Undef**. Dacă nu, dacă există o polaritate aleasă de utilizator, acesta este aignată. În caz contrar, dacă variabila **rnd_pol** este asig-nată se alege o polaritate aleatoare. Dacă nici una dintre condițiile de mai sus nu sunt împlinite se asignează o polaritate predefinită **polarity[next]**. Algoritmul folosit de această metodă este reprezentat în cadrul diagramei de activitate prezentată în Figura 2

Metoda propagate() - Metoda este responsabilă pentru propagarea unitate, atribuirea de valori bazată pe clauzele învățate pentru a reduce domeniul de căutare a unui model. În variabila **Lit p** se reține literalul curent de la care se pleacă cu propagare. Acesta este extras din coada **trail** care este parcursă cu

ajutorul variabilei `qhead`. Pentru fiecare literal, funcția `propagate()` parcurge fiecare clauză. Dacă aceasta este deja satisfăcută, atunci se trece peste ea. Dacă nu este, se caută un literal care nu are o valoare atribuită pentru a încerca să satisfacă clauza. Dacă nu se găsește nici un literal care să satisfacă clauza se aplică din nou propagarea unitate (linia 554). În cadrul acestei funcții se mai execută și verificarea unor noi conflicte, iar dacă unul este găsit se va asigura în variabila `conf1` referința către acest conflict (liniile 545-552). O reprezentare a acestor pași realizați de algoritm se poate observa în cadrul Figurii 3

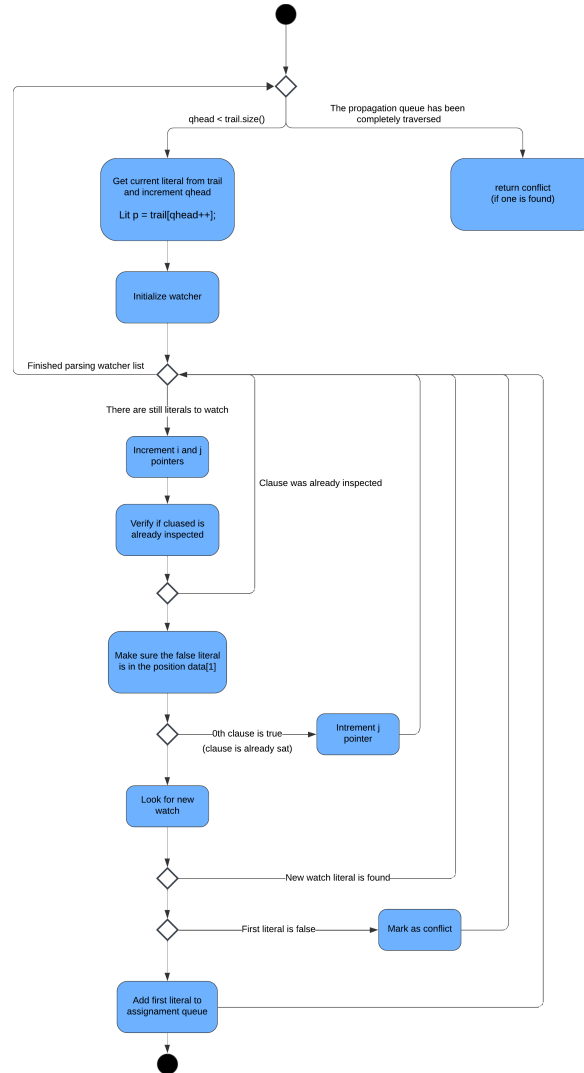


Figura 3: Diagrama de activitate a metodei *propagate*

Metoda *analyze*(*CRef confl*, *vec*<*Lit*>*ℰ* *out_learn*, *int*ℰ *out_btlevel*) - Analizează conflictul transmis ca parametru pentru a produce clauza care a produs acest conflict. Clauza găsită este adăugată în mulțimea de clauze învățate pentru a optimiza procesul de căutare a unui model, aplicându-se propagarea unitate peste această mulțime pentru a asigura valori unor literali bazate pe implicațiile rezultate din aceste clauze, ajutând la evitarea conflictelor întâlnite deja în procesul căutării.

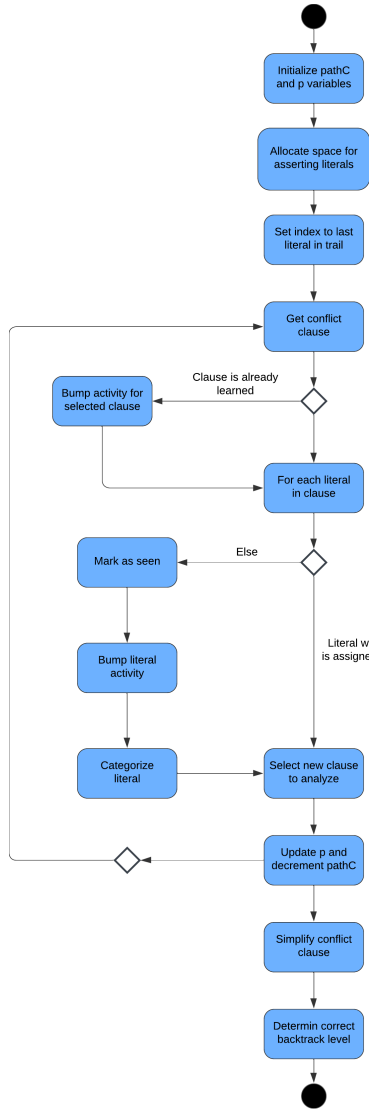


Figura 4: Diagrama de activitate a metodei *analyze*

Metoda începe prin inițializarea variabilelor **pathC** și **p**(literalul curent analizat). În cadrul buclei **do-while** se analizează clauza de conflict curentă, iar dacă aceasta este deja învățată, i se crește activitatea. Pentru fiecare literal al clauzei dacă nu mai fost văzut până în acel moment și se găsește pe un nivel de decizie mai mare decât 0, atunci este marcat ca văzut. Dacă nivelul pe care se găsește acest literal este mai mare sau egal cu nivelul de decizie, se incrementează variabila **pathC**, iar dacă nu literalul este adăugat în variabila **out_learned**. După acest pas se caută următoarea clauză care va trebui analizată pe baza următorului literal din coada **trail**. Pentru acesta se va găsi clauza care l-a produs și va fi asignată către **conf**, iar apoi se va decrementa variabila **pathC**. Această buclă se oprește atunci când ajunge variabila **pathC** 0. După terminarea buclei, se realizează simplificarea clauzei, urmată de găsirea nivelului corect de backtracking. Pentru o înțelegere mai ușoară a algoritmului se poate vedea Figura 4

*Metoda **search(int nof_conflicts)*** - Metoda începe prin declararea și inițializarea variabilelor pentru nivelul de backtrack, variabilei **conflictC** și clauzelor învățate, apoi se începe un ciclu infinit în care se încearcă găsirea unui model. Pentru o înțelegere mai ușoară a fluxului prin care algoritmul trece puteți să consultați Figura 5. Ciclu începe prin căutarea unei clauze de conflict cu ajutorul metodei **propagate()**. În cazul găsirii unui conflict se verifică dacă nivelul de decizie este 0, iar dacă această condiție este adevărată se returnează **l_False**, deducându-se că formula este satisfiabilă. Dacă acesta este diferit de 0, se resetează lista de clauze învățate și folosind metoda **analyze()** pentru a o repopula cu noile clauze învățate care iau în considerare conflictul găsit. După analiză se revine la ultimul nivel găsit aplicând metoda **cancelUntil()**.

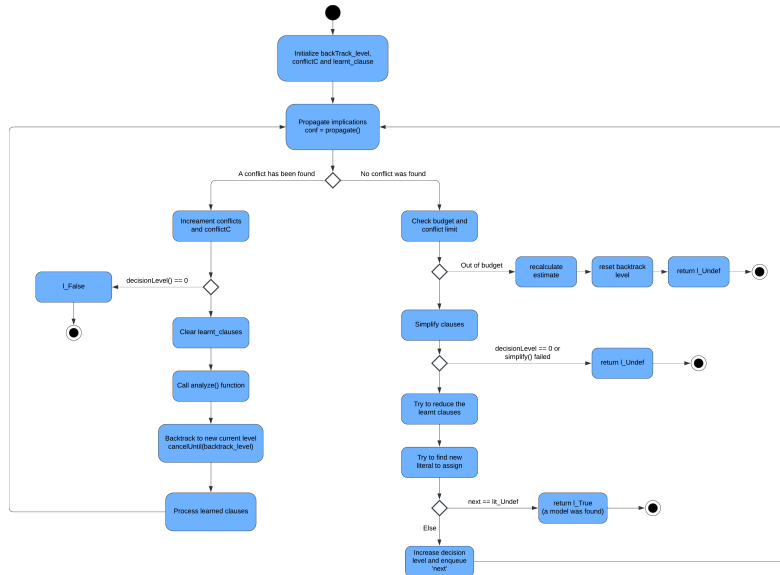


Figura 5: Diagrama de activitate a metodei *search*

Dacă nu este găsit nici un conflict verificăm dacă ne aflăm în limitele setate, adică limita de conflicte, dacă acesta există și limitele de resurse (timp, memorie). Dacă una oricare dintre limită este depășită se va return `l_Undef`, indicând că nu s-a putut studia complet formula în limitările setate. Dacă se trece de această verificare se încearcă simplificarea clauzelor. Dacă ne aflăm pe nivelul 0 și nu se poate realiza nici o simplificare atunci se va returna `l_False`. Următoarea acțiune care se realizează este reducerea clauzelor învățate dacă numărul acestora depășește limita setată.

După realizarea verificărilor menționate mai sus, se realizează o nouă decizie. Inițial se încearcă asignarea unei valori indicate de utilizator asupra unui literal. Dacă aceste presupuneri nu există se încearcă asignarea folosind metoda `pickBranchLit()`. Dacă și după această încercare de asignare nu a putut fi aleasă nici o valoare înseamnă că am găsit un model, deci vom returna `l_True`. Dacă s-a ales o valoare pentru un literal, incrementăm nivelul de decizie și adăugăm noul literal în coadă.

Identificarea algoritmilor de enumerare, DPLL sau CDCL În urma analizei realizate asupra codului am putut deduce că aplicația implementează algoritmul CDCL pentru a optimiza căutarea unui model. Acest algoritm are la bază algoritmul DPLL, care presupune la fiecare pas alegerea unei decizii, un literal și o polaritate pentru acesta, și se încearcă să se determine noi literalii analizând restul clauzelor. Dacă se găsește un conflict, se urmăresc deciziile care au dus la acest conflict și se reiau luarea deciziilor de la cel mai mic nivel. Optimizările aduse de CDCL presupun învățarea literalilor care au provocat conflicte pentru a nu mai repeta din nou acele conflicte. Putem observa acest comportament în cadrul programului MiniSAT putem observa cum în metoda `search()` în cazul găsirii unui conflict se apelează metoda `analyze`, care actualizează clauzele învățate prin propagarea fiecărui literal din lista de conflicte. Aceste metode sunt prezentate mai detaliat în cadrul Subsecțiunii 3.3

4 Descrierea familiei *scheduling*

În această familie de probleme, sunt abordate mai multe probleme de planificare. În fișierul `2a15a30186afdad41a49c5c5366d01be-Timetable_C_392_E_62_Cl_26_S_28.cnf` se abordează o problemă de **planificare a unui orar școlar**. În [18], autorii propun o abordare de realizare unui orar supus doar unor constrângeri tari care trebuie satisfăcute. Un curs este definit de o clasă (un grup de studenți) și o disciplină, iar fiecare curs trebuie să fie predat de un profesor, să aibă o sală și un anumit număr de locuri în sală. În plus, două cursuri predate de același profesor nu se pot desfășura în același timp, nu pot avea loc două cursuri în aceeași sală în același timp, profesorii pot avea intervale în care nu sunt disponibili, trebuie respectat numărul de ore alocate unui curs, iar orele cursului trebuie să fie apropiate, profesorii pot să nu fie calificați să predea acest curs și orele unui profesor trebuie să se întindă pe un număr limitat de zile.

Autorii doresc să găsească pentru fiecare curs al fiecărei clase un profesor care să predea cursul, o sală în care să aibă loc acel curs și intervalele orare ale acestuia. Aceștia au denumit variabilele, seturile și constantele necesare rezolvării problemei, dar și constrângerile prezentate mai sus și le-au scris sub formă de expresie logică. Pentru această problemă au fost generate 20 de instanțe, rulate utilizând solverul **MapleLCMDistChronoBT** cu o limită de timp de 5000 de secunde și limită de memorie de 24 GB. Unul dintre cazuri a fost rulat folosind 272 de cursuri, 52 de profesori, 18 clase și 32 de săli, și s-a obținut un rezultat **satisfiabil** într-un timp de 80.65 secunde.

Un alt subiect abordat în această familie de probleme este **programarea optimă a sarcinilor paralele**. Autorii lucrării [1] definesc problema de planificare $P||C_{max}$ drept un set de sarcini n cu timpi de execuție cunoscuți w distribuite la un set de mașini identice (m) și propun o abordare neconstrânsă a acestei probleme. Aceștia au ordonat în ordine descrescătoare sarcinile (j), în funcție de timpul de execuție și s-a căutat o modalitate de a asigna fiecărui procesor (p) o sarcină. S-a denumit câte o variabilă $a_{j,p}$ și s-au scris clauzele sub formă de propoziții logice, astfel încât unui procesor să îi fie asignată o singură sarcină. La final s-a verificat ca suma ponderilor atribuite fiecărui procesor nu depășește limita superioară considerată pentru fiecare procesor, care este echivalent cu constrângerea pseudo-booleană (PBC). Aceștia au redus această PBC la diagrama de decizie binară ordonată redusă (BDD). Diagrama reprezintă combinațiile de sarcini ce pot fi atribuite unui proces, iar datorită faptului că este binară, aceasta poate produce doar răspunsuri *true* sau *false*. În plus a fost forțată unei sarcini primului procesor care are încărcare identică, apoi a fost forțată alocarea unei sarcini nealocate unui procesor căruia i-a rămas spațiu nealocat. Au fost considerate 11 instanțe cu un număr de sarcini cuprins între 20 și 220 și $n/m \in [2, 3]$, care au fost rulate utilizând solver-ul **KISSAT** cu o limită de timp de 500 de secunde, obținând pentru fiecare dintre acestea un rezultat nesatisfiabil. În urma acestei rulări au fost alese alte 9 instanțe care au produs rezultatul satisfiabil într-un timp cuprins între 13 și 54 de secunde.

În lucrarea [19] este definită problema **Worker Dispatching Problem (WDP)**. Această problemă se referă la angajarea muncitorilor din fabrici în realizarea unor sarcini. În această problemă se consideră n *fabrici*, iar pentru fiecare fabrică F_i există k *muncitori* care pot fi atribuiți unui proiect comun. Fiecare proiect oferă m *locuri de muncă (sau sarcini)*, iar fiecare muncitor are un set de aptitudini. Din cauza numărului redus de muncitori, fiecărui muncitor îi poate fi asignat cel mult o sarcină compatibilă cu aptitudinile acestuia. Așadar, se caută o soluție fezabilă de atribuire a sarcinilor, ținându-se cont de faptul că fiecare sarcină trebuie să fie realizată de un muncitor și fiecare fabrică oferă un muncitor căruia îi poate fi atribuită cel mult o sarcină. Au fost ignorate aranjamentele imposibile. S-a observat că în încercările în care sunt un număr aproximativ egal de fabrici și sarcini, iar fiecare fabrică oferă 2 muncitori, există o probabilitate mai mare ca sarcinile să fie realizate. Fișierele rezultate în această categorie au fost rulate utilizând **MiniSat** utilizând un timp CPU de peste un minut.

O altă problemă abordată în această familie este **realizarea unui orar constrâns de timp pentru turneele Double Round Robin (2RR)**, descrisă de către autorii lucrării [12]. Pentru această problemă se definesc n echipe și s sloturi de timp. În sportul Double Round Robin, fiecare echipă joacă odată în fiecare rundă și fiecare echipă joacă de două ori (pe terenul propriu și pe terenul adversarului) împotriva fiecărei dintre celelalte echipe, rezultând în $s = 2 \cdot (n-1)$ sloturi de timp. O pauză are loc atunci când o echipă 2 jocuri consecutive, iar din dorința de a minimiza acest număr, numărul de pauze este redus la $s = 2 \cdot (n-2)$. Această problemă a fost abordată de către International Timetabling Competition 2021 (ITC 2021), utilizând instanțele problemelor furnizate de organizatorii formatului RobinX bazat pe XML, folosind un set de constrângeri hard și soft. Aceștia au propus *constrângeri* de tip *pseudo-boolean (PB)*, iar pentru rezolvarea acestora s-a utilizat **clasp**, un solver pseudo-boolean. În comparație cu alte solvere care au folosit o căutare locală personalizată sau soluții comerciale precum Mixed Integer Programming (MIP), cum ar fi **Gurobi** și **CPLEX**, acesta nu a oferit cele mai calitative rezultate, dar din punct de vedere al timpului, oferea răspunsuri în mai puțin de 10 minute. Au fost utilizate variabile de decizie, care erau adevărate doar dacă o echipă joacă "acasă" împotriva altei echipe în acel slot. În plus fiecare echipă trebuie să joace exact odată în acel slot, iar fiecare meci (pe terenul propriu sau pe terenul adversarului) are loc o singură dată. Pentru a produce benchmark-ul au utilizat *Reprobate* pentru generarea codificărilor pentru formatul RobinX, fără a folosi constrângerile soft. Apoi au folosit *pbencoder* pentru a le transforma în instanțe SAT. Instanțele au fost împărțite în 3 categorii: cu dificultate mică (rulate cu **MiniSAT 2.2.1** în mai puțin de un minut), cu dificultate medie (rulate cu **clasp 3.3.5** în mai puțin de o oră) și cu dificultate mare (cele care nu au putut fi rulate cu clasp în mai puțin de o oră). Pentru competiția SAT au fost generate instanțe cu doar o constrângere pentru problema Double Round Robin, unde numărul de echipe a fost cuprins între 4 și 20. Pentru acestea există 3 variații, pentru $k \in [2, 5]$:

1. Numărul de pauze să fie maxim $k \cdot (n-2)$, $k=2$ fiind numărul minim de pauze;
2. Numărul de pauze să fie maxim $k \cdot (n-2)$, dar nu trebuie să existe secvențe de 3 sloturi consecutive care conțin pauze. Această constrângere suplimentară a îmbunătățit performanța minimizării numărului de pauze;
3. Numărul de pauze să fie maxim $k \cdot (n-2) - 1$. În comparație cu celelalte variații, aceasta a fost nesatisfiabilă.

5 Rezultatele experimentale

Pentru familia *scheduling*, au fost rulate fișierele specifice acestei familii. Pentru rularea testelor, a fost setat un timp CPU limită de *1728 de secunde*. Pentru a realiza rulările, denumirile tuturor fișierelor au fost înlocuite cu numere din intervalul $[1, 50]$. Aceste denumiri se regăsesc în Figura 6, pe prima coloană fiind fișierele notate cu numere impare, iar pe a doua cele notate cu numere pare.

```

07c50b89d97a55b6e1629999e445a2be-ctl_4201_555_unsat_pre.cnf
07e6413459f2b613498a719125b6239-3037_10_mdd_bm1.cnf
09d7add3bf3b75c5d1023a92e752989a-Break_unsat_04_03.xml.cnf
0ac5aaf6db6a0e4ec3a0e8e5d0108f1-pcmx-scheduling-m13-2011-12813-UNSAT.sanitized.cnf
0d3fca0b89b6a921038a39a424d4f-3045_10_gmto_b.cnf
11c5f32cedfb10a47e0a88b4867c1b-Break_trip6-20_36.xml.cnf
11af62d7a3d817c3149da432eece66da8-worker_550_550_550_0_3.cnf
207b28c3cefc55b143259222cbfd4962-pcmx-scheduling-m37-28831-324346-SAT.sanitized.cnf
22b4c412c811872d6ba5078106aeb6c-3037_10_rggt_b.cnf
29a61b1317804fb897e64237613d42bb-worker_30_60_25_0_8.cnf
2a1ba3016a6dnd41a49c5c5366d07be-Timetable_C_392_E_02_CL_26_S_28.cnf
31a788a12dd8b8c4ac20e77d53abaa23-pcmx-scheduling-m40-26287-324155-SAT.sanitized.cnf
32585e66749f42e161e94fc49fd64750-pcmx-scheduling-m26-6398-62377-UNSAT.sanitized.cnf
3593875d75d6a83a3c328c5426c1b5-Break_18_32.xml.cnf
3988a60cae9316775c5c8d2a347d5859-Break_08_24.xml.cnf
43a492bcd6f57028b756897a17b7d44-pb_300_0_0_lb_07.cnf
44c25de7963c45e92a2407ad8399e8b-Break_20_72.xml.cnf
5432e4d4448f0adbd96647db1750c80-pcmx-scheduling-m24-24102-255206-SAT.sanitized.cnf
576ebc133f2c466c2dec98792721e1f1-3037_9_rggt_b.cnf
5952b918b73dab531c603ac85d26f-ctl_4291_567_9_unsat.cnf
5a088a62a13b0a358762e2e889e4f-ITC2021_Middle_4.xml.cnf
6b5cfab02e5c12a50a53bc7d7cc42c7-pcmx-scheduling-m15-2352-13561-SAT.sanitized.cnf
6ea72bdcc88bd1a924029f7b81aec5-atco_enc1_opt1_10_15.cnf
62dcfb79f9cecf5618c9da00311326c-pcmx-scheduling-m13-1655-9604-UNSAT.sanitized.cnf
6fc528fc3d0fd5a2c50992feb8f0357-pcmx-scheduling-m24-17855-226744-SAT.sanitized.cnf
74f145bb935650f5c982d7eecc6967945-pcmx-scheduling-m43-38782-385402-SAT.sanitized.cnf
71c7da531539f3be4a1e0e916913086-pcmx-scheduling-m19-2974-16501-UNSAT.sanitized.cnf
77109dce621ef361a72b3e8cee9a962-Break_unsat_06_07.xml.cnf
7fca438c38a5b7a70e38778ec145cfe0-ctl_4291_567_2_unsat-sc2013.cnf
8a3c30c43a6c4c3d95e181e383883c-3037_1_mdd_b.cnf
9add280b1562ee7dae44b303b8fed233-Break_unsat_18_31.xml.cnf
9e85aad5707405fb349c6c5548040c14-pcmx-scheduling-m35-32274-371389-SAT.sanitized.cnf
a08e66296d00480e9ccadd79fab904-3045_4_gmto_b.cnf
b087c195e5b7b191d3a701aa0aa8cf1-Break_unsat_14_23.xml.cnf
bd12a2b66110451b050bd5d2945b1854-ITC2021_Early_10.xml.cnf
bd86c22b2a38c5d438459c17eb15814-pcmx-scheduling-m11-1517-6802-UNSAT.sanitized.cnf
be64114784a3c879886dda807cdc07-3037_10_mdd_b.cnf
bfadd34e71516e95d0ba2a3a8fb2ba03-pcmx-scheduling-m19-10199-62102-UNSAT.sanitized.cnf
c801a020a6c8bc3c287fea495203b114-worker_20_40_20_0_95.cnf
ca14adcb9296a7b31d7815c2ad16d0f1-ITC2021_Early_3.xml.cnf
cc30a4eedebcf9c2cc1108a4d98781246-3037_9_mdd_bm1.cnf
d02fae1e84cfbe799c792ba23708f96-exam_flat_0_04_2018_3.cnf
d7627176604d5b8e8f3d2f5d14d8ee99-worker_40_80_40_0_9.cnf
da929c2ae0de5e59878ad87c4323e581-pcmx-scheduling-m12-8049-55035-SAT.sanitized.cnf
dafc03aa784fee849ebca7b64230558-pcmx-scheduling-m30-14113-167638-UNSAT.sanitized.cnf
e481697a93698a323f99024f19e2687-3037_1_gmto_b.cnf
eda4aa84aeb305468396a82a8bba5e-pb_300_0_0_lb_17.cnf
f7e855b18105170b0718086d3f5b5923-3045_10_rggt_b.cnf
fae08ac5b6d7eeb4f56f1cfac6c51703-3045_4_mdd_bm1.cnf
fb45d41807cbbbe689de282c1310798-Break_unsat_16_27.xml

```

Figura 6: Fișiere din familia *scheduling*.

Testele au fost rulate pe diferite configurații:

1. Laptop **Macbook Air**:
 - Procesor **Apple M1**
 - Memorie RAM **8GB**
 - Sistem de operare **MacOS Sonoma 14.4.1**
2. Laptop **Lenovo V310-15IKB**:
 - Procesor **Intel Core i7-7500U**
 - Memorie RAM **8GB**
 - Sistem de operare **Ubuntu-22.04 prin WSL⁴**

Familia *scheduling* cuprinde fișiere care conțin un număr variabile cuprins între **221** și **1275631** și cu un număr de clauze cuprins între **1084** și **10295409**.

Rezultatele rulării acestor fișiere, folosind MiniSat, sunt prezentate în Tabelul

1. În urma rulării, am primit răspuns SAT sau UNSAT pentru **19/50 fișiere** în cazul primei configurații și pentru **14/50 fișiere** în cazul celei de-a doua configurație, având următoarele valori pentru timpul CPU și memorie:

- Cele mai mici valori pentru timpul CPU și memoria utilizată pentru care a fost determinat un rezultat Pentru fișierul cu **221 variabile** și **1084 clauze** (cele mai mici valori), a fost obținut rezultatul **UNSAT**, utilizând **0.009421 secunde** timp CPU și **4.85 MB** de memorie în cazul primei configurații și **0.006469 secunde** timp CPU și **5.97 MB** de memorie în cazul celei de-a doua.
- Timpul CPU și memoria utilizată pentru fișierul cu cel mai mare număr de variabile Pentru fișierul cu **145943 variabile** și **625454 clauze**, a fost obținut rezultatul **SAT**, utilizând **20.1524 secunde** timp CPU și **181.53 MB** de memorie în cazul primei configurații și **73.6448 secunde** timp CPU și **182.39 MB** de memorie în cazul celei de-a doua.
- Timpul CPU și memoria utilizată pentru fișierul cu cel mai mare număr de clauze Pentru fișierul cu **93713 variabile** și **10295409 clauze**, a fost

⁴ Windows Subsystem for Linux

obținut rezultatul **SAT**, utilizând **58.533 secunde** timp CPU și **1996.39 MB** de memorie în cazul primei configurații și **146.975 secunde** timp CPU și **1833.28 MB** de memorie în cazul celei de-a doua.

- Cele mai mari valori pentru timpul CPU și memoria utilizată pentru care a fost determinat un rezultat Pentru fișierul cu **145862 variabile** și **624664 clauze**, a fost obținut rezultatul **SAT**, utilizând **379.05 secunde** timp CPU și **194.04 MB** de memorie în cazul primei configurații și **1235.91 secunde** timp CPU și **1593.7 MB** de memorie în cazul celei de-a doua.

Tabela 1: Rezultatele testelor experimentale.

Scheduling								
			Configurație 1			Configurație 2		
Test	Variabile	Clauze	Timp CPU	Memorie	Răspuns	Timp CPU	Memorie	Răspuns
1	14756	141683	1109.72 s	200.94 MB	UNSAT	1826.18 s	226.49 MB	INDET
2	38781	362841	1659.87 s	381.07 MB	INDET	1873.77 s	1082.81 MB	INDET
3	14400	63874	17.3344 s	33.80 MB	UNSAT	38.1849 s	33.24 MB	UNSAT
4	2973	15516	1648.76 s	146.58 MB	INDET	1880.09 s	68.54 MB	INDET
5	221	1084	0.009421 s	4.85 MB	UNSAT	0.006469 s	5.97 MB	UNSAT
6	1101	5036	0.422917 s	9.49 MB	UNSAT	0.678463 s	6.23 MB	UNSAT
7	2010	11953	1465.29 s	141.27 MB	INDET	1818.18 s	53.30 MB	INDET
8	17850	149132	946.04 s	195.01 MB	UNSAT	1881 s	202.65 MB	INDET
9	14560	48075	1543.39 s	179.64 MB	INDET	1843.72 s	126.55 MB	INDET
10	21288	87245	694.386 s	161.01 MB	SAT	1885.32 s	134.71 MB	INDET
11	49869	264805	1686.52 s	463.18 MB	INDET	1852.69 s	1035.09 MB	INDET
12	37005	187726	1677.85 s	806.57 MB	INDET	1789.65 s	475.77 MB	INDET
13	93713	10295409	58.5334 s	1996.39 MB	SAT	146.975 s	1833.28 MB	SAT
14	32273	361039	1330.87 s	798.20 MB	INDET	1784.44 s	1630.84 MB	INDET
15	28830	320272	1621.04 s	1697.41 MB	INDET	1856.52 s	1659.16 MB	INDET
16	14128	46571	1717.94 s	161.27 MB	INDET	1784.44 s	98.07 MB	INDET
17	14174	66704	13.9012 s	43.11 MB	SAT	33.8056 s	29.14 MB	SAT
18	22365	83896	1715.57 s	298.08 MB	INDET	1795.28 s	252.23 MB	INDET
19	1015	21642	1706.28 s	327.14 MB	INDET	1829.38 s	271.76 MB	INDET
20	249117	764929	1314.89 s	2797.59 MB	INDET	1804.97 s	1710.56 MB	INDET
21	282525	1743751	449.656 s	2999.46 MB	INDET	1830.74 s	4562.01 MB	INDET
22	1516	6635	1122.38 s	121.11 MB	UNSAT	1811.21 s	40.80 MB	INDET
23	26286	319289	1691.84 s	823.04 MB	INDET	1834.96 s	1640.22 MB	INDET
24	14945	66518	14.4803 s	38.26 MB	SAT	34.4257 s	33.75 MB	SAT
25	6397	60575	246.293 s	96.22 MB	UNSAT	569.099 s	96.45 MB	UNSAT
26	10198	61395	1673.77 s	198.71 MB	INDET	1814.36 s	153.00 MB	INDET
27	37028	187771	1705.58 s	1323.73 MB	INDET	1841.8 s	716.64 MB	INDET
28	656	11934	0.011122 s	5.35 MB	SAT	0.007044 s	6.55 MB	SAT
29	3890	14187	0.068435 s	6.82 MB	SAT	0.064183 s	7.41 MB	SAT
30	29478	125712	0.106474 s	12.52 MB	SAT	0.170763 s	16.52 MB	SAT
31	145943	625454	20.1524 s	181.53 MB	SAT	73.6448 s	182.39 MB	SAT
32	19502	82348	1709.56 s	191.89 MB	INDET	1815.89 s	153.42 MB	INDET
33	53844	270141	1708.94 s	1572.19 MB	INDET	1832.5 s	1247.98 MB	INDET
34	1275631	1974386	1707.63 s	1204.72 MB	INDET	1821.6 s	686.98 MB	INDET

35	24101	254904	1684.74 s	829.19 MB	INDET	1820.72 s	1059.04 MB	INDET
36	2508	87544	1702.79 s	834.93 MB	INDET	1820.79 s	1702.12 MB	INDET
37	18913	85586	1705.65 s	183.61 MB	INDET	1837.52 s	149.83 MB	INDET
38	8048	54850	1707.95 s	281.02 MB	INDET	1837.36 s	180.82 MB	INDET
39	17850	154655	223.214 s	131.86 MB	UNSAT	544.334 s	158.11 MB	UNSAT
40	14112	166497	1381.68 s	259.28 MB	INDET	1829.94 s	229.40 MB	INDET
41	58907	193418	1675.12 s	400.32 MB	INDET	1851.66 s	690.25 MB	INDET
42	21015	69505	1138.31 s	152.57 MB	SAT	1841.53 s	156.52 MB	INDET
43	2351	13401	1415.36 s	180.14 MB	INDET	1842.87 s	123.18 MB	INDET
44	145862	624664	379.05 s	194.04 MB	SAT	1235.91 s	1593.70 MB	SAT
45	46759	250518	86.6242 s	99.90 MB	SAT	265.449 s	210.96 MB	SAT
46	14751	63112	1707.47 s	174.44 MB	INDET	1842.64 s	119.05 MB	INDET
47	1654	8801	1678.18 s	156.80 MB	INDET	1869.49 s	43.13 MB	INDET
48	13043	51999	1380.45 s	148.19 MB	INDET	1841.23 s	97.95 MB	INDET
49	17854	226553	1694.31 s	800.26 MB	INDET	1871.49 s	697.38 MB	INDET
50	31113	127844	1638.65 s	1227.83 MB	INDET	1851.76 s	380.50 MB	INDET

Din punct de vedere statistic, pentru prima configurație au fost obținute **11/19** răspunsuri **SAT** și **8/19** răspunsuri **UNSAT**, iar pentru a doua configurație au fost obținute **9/14** răspunsuri **SAT** și **5/14** răspunsuri **UNSAT**.

Corelația dintre numărul de variabile și timpul de execuție este prezentat în Figura 7a pentru primele 25 de fișiere cu valori cuprinse între [221, 17850] în ordine crescătoare și în Figura 7b pentru următoarele 24 de fișiere cu valori cuprinse între [17854, 282525] în ordine crescătoare. Fișierul cu cel mai mare număr de variabile a fost exclus din aceste grafice, deoarece numărul de variabile era cu mult mai mare decât celelalte valori.

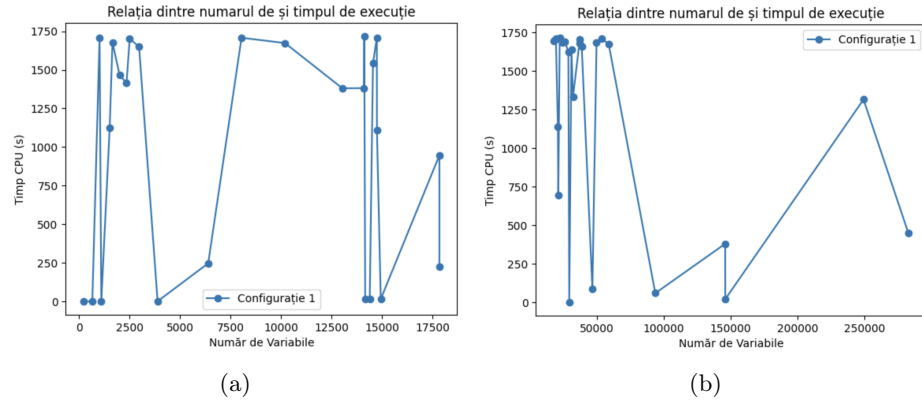


Figura 7: Corelarea numărului de variabile cu timpul de execuție. (a) Primele 25 de valori în ordine crescătoare; (b) Ultimele 24 de valori în ordine crescătoare

Corelația dintre numărul de clauze și timpul de execuție este prezentat în Figura 8a pentru primele 25 de fișiere cu valori cuprinse între [1084, 87245] în

ordine crescătoare și în Figura 8b pentru următoarele 24 de fișiere cu valori cuprinse între $[87544, 1974386]$ în ordine crescătoare. Fișierul cu cel mai mare număr de clauze a fost exclus din aceste grafice, deoarece numărul de clauze era cu mult mai mare decât celelalte valori.

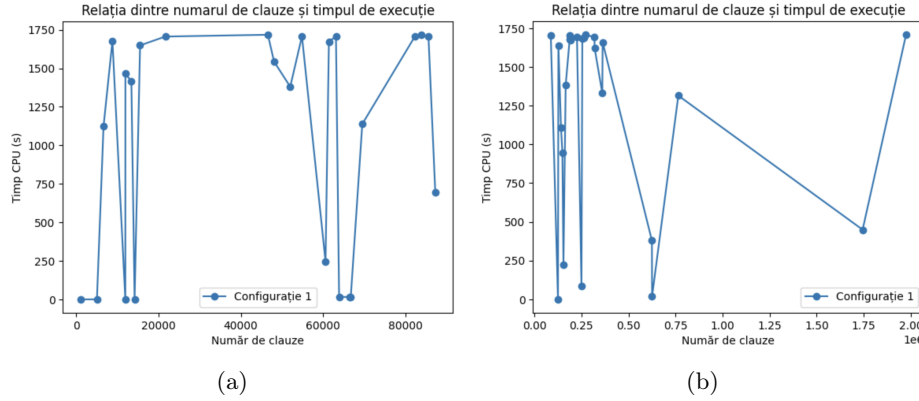


Figura 8: Corelarea numărului de clauze cu timpul de execuție. (a) Primele 25 de valori în ordine crescătoare; (b) Ultimele 24 de valori în ordine crescătoare

Din aceste grafice se poate observa că nu există o corelație directă între timpul de execuție și numărul de variabile sau numărul de clauze. Acest timp este influențat de modul în care CDCL găsește conflictele și învață clauzele care produc aceste conflicte. Totodată, depinde și de numărul de clauze excluse după găsirea conflictelor, deoarece un număr de clauze mai mic presupune o rulare mai rapidă.

5.1 Crearea de programe ajutătoare

Pentru a avea o mai bună înțelegere a familiilor am realizat un program Python ce se află în folderul *pythonprogram* din proiectul prezent pe GitHub. Acest program utilizează Web Scraping pentru a colecta și număra fișierele pentru fiecare familie, stocând într-un folder câte un fișier cu numele familiei și o listă cu hash-urile pentru descărcarea fișierelor din acea familie.

Pe lângă programul Python, am avut o serie de programe scrise folosind Bash pentru Linux. Fișierul *downloadedfile.sh* preia toate fișierele pentru fiecare familie și le descarcă într-o locație dată. La rularea programului se creează o altă locație cu fișierele descărcate, dezarhivate și redenumite. Pentru rularea unei familii am utilizat un timp de rulare prestabilit (de 24 de ore împărțit la numărul de fișiere) pentru fiecare fișier. Am utilizat un program simplu Bash care primește ca argument familia pentru care urmează să se execute programul MiniSat iar datele de ieșire se păstrău într-un fișier de output pentru fiecare fișier denumit *runfamilyfiles.sh*

6 Provocări

Provocările întâmpinate pe parcurs au fost unele diverse și datorate mai multor cauze.

1. Instalarea MiniSat Modulurile variate de Instalare/rulare ale MiniSat pe diferite sisteme de operare a fost prima problemă întâmpinată. Cel mai bun mod de a testa MiniSat descoperit de noi a fost prin intermeniul WSL sau chiar folosind o configurație exclusivă Linux. Pe configurația Windows, rezulatele nu au fost deloc promițătoare precum cele de pe MacOS sau Linux.
2. Optimizarea codului Introducerea unui mecanism de multithreading, pentru a permite mai multor fire de execuție să rezolve instanțele de SAT în paralel. Modificarea heuristicii de decizie, pentru a face alegerea variabilelor mai eficientă în timpul propagării.
În timpul implementării, am întâmpinat dificultăți majore în compilarea proiectului. După modificarea fișierului Solver.h, compilarea s-a oprit din cauza unor conflicte între declarațiile existente și noile definiții introduse. Conflictele plecau în general de la modul cum a fost gândit programul să ruleze pe medii special concepute pentru acest gen de programe, iar modificarea și apoi rularea acestuia pe calculatoare obișnuite este considerabil mai grea. Ideile de optimizare aduceau bonus de performanță doar în anumite cazuri, modul deja existent al MiniSat fiind unul universal optim.
3. Programe ajutătoare Programele ajutătoare descrise în Secțiunea 5. Acele programe au ajutat la alegerea familiei pentru benchmark, pregătirea fișierelor și rularea MiniSat cu anumite constrângeri.

7 Concluzie

În urma cercetării și experimentelor realizate în acest proiect, am obținut rezultate relevante privind performanța solverului SAT MiniSat, utilizând benchmark-uri din familia *scheduling* și rulând testele pe trei sisteme de operare diferite, Windows, Linux și MacOS. Rezultatele noastre au evidențiat diferențe semnificative între performanțele obținute pe cele două platforme. Astfel, utilizarea MiniSat pe platforma WSL a demonstrat o consistență mai mare în timp de execuție și consum de resurse, în special pentru fișiere cu soluție imediată (timp de execuție sub 0.01 secunde). Pe de altă parte, platforma MacOS a înregistrat performanțe competitive pentru majoritatea fișierelor din familia Scheduling. Motivația inițială a proiectului, bazată pe lipsa unor lucrări clare care să documenteze diferențele de performanță între sistemele de operare, s-a dovedit validă. Rezultatele noastre susțin ipoteza că alegerea platformei poate influența considerabil eficiența solverelor SAT, iar această alegere ar trebui să fie fundamentată pe tipul specific de probleme și resurse disponibile. În concluzie, acest studiu oferă o contribuție valoroasă în înțelegerea performanțelor solverelor SAT în funcție de platforma utilizată și evidențiază importanța adaptării configurațiilor solverului la specificațiile benchmark-urilor abordate. De asemenea, oferă direcții pentru cercetări viitoare, cum ar fi explorarea detaliată a optimizărilor arhitecturale și software care pot îmbunătăți performanța solverelor SAT în contexte specifice.

Bibliografie

1. Matthew Akram and Dominik Schreiber. Optimal parallel task scheduling via sat. In *Proceedings of SAT Competition 2024 : Solver, Benchmark and Proof Checker Descriptions*, pages 30–31. Helda University of Helsinki, 2024.
2. Franc Brglez, XY Li, and MF Stallmann. On sat instance classes and a method for reliable performance experiments with sat solvers. *Annals of Mathematics and Artificial Intelligence*, 43:1–34, 2005.
3. Martin Davis, George Logemann, and Donald Loveland. A computing procedure for quantification theory. *Journal of the ACM (JACM)*, 7(3):201–215, 1960.
4. Martin Davis and Hilary Putnam. A machine program for theorem-proving. In *Communications of the ACM*, volume 5, pages 394–397. ACM New York, NY, USA, 1962.
5. Niklas Een and Niklas Sörensson. Minisat v2. 0 (beta). *Solver description, SAT race*, 2006, 2006.
6. Niklas Een and Niklas Sörensson. Minisat — a sat solver. <https://minisat.se>. Accessed: 2025-01-25.
7. Niklas Een and Niklas Sörensson. An extensible sat-solver. In *International conference on theory and applications of satisfiability testing*, pages 502–518. Springer, 2003.
8. Tobias Fuchs, Jakob Bach, and Markus Iser. Active learning for sat solver benchmarking. In Sriram Sankaranarayanan and Natasha Sharygina, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 407–425, Cham, 2023. Springer Nature Switzerland.
9. Carla P. Gomes, Bart Selman, and Henry Kautz. Satisfiability solvers. *Handbook of Knowledge Representation*, 3:89–134, 2008.
10. Marijn JH Heule, Markus Iser, Matti Järvisalo, and Martin Suda. Proceedings of sat competition 2024: Solver, benchmark and proof checker descriptions. 2024.
11. Jinbo Huang et al. The effect of restarts on the efficiency of clause learning. In *IJCAI*, volume 7, pages 2318–2323, 2007.
12. Martin Mariusz Lester. Sports timetabling sat benchmarks. In *Proceedings of SAT Competition 2022 : Solver and Benchmark Descriptions*, pages 68–69. Helda University of Helsinki, 2022.
13. Joseph Y-T. Leung and Jørgen D. Nielsen. *Handbook of Scheduling: Algorithms, Models, and Performance Analysis*. CRC Press, 2004.
14. Joao P Marques-Silva and Karem A Sakallah. Grasp: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.
15. Lawrence Ryan. Efficient algorithms for clause-learning sat solvers. 2004.
16. SAT Competition Committee. SAT Competition 2024 Rules, 2024.
17. Niklas Sörensson and Niklas Een. Minisat v1.13-a sat solver with conflict-clause minimization. In *Proc. SAT*, volume 8, page 53, 2005.
18. Rodrigue Konan Tchinda and Clémentin Tayou Djamegni. A sat encoding of school timetabling. In *Proceedings of SAT Competition 2020: Solver and Benchmark Descriptions*, pages 97–99. Helda University of Helsinki, 2020.
19. Xindi Zhang, Zhihan Chen, and Shaowei Cai. Factory worker dispatching problem. In *Proceedings of SAT Competition 2022 : Solver and Benchmark Descriptions*, pages 82–83. Helda University of Helsinki, 2022.

8 Împărțirea responsabilităților în echipă

1. Berindeie Adrian

- script pentru descărcare fișiere pe familii
- script pentru pregătirea și rularea benchmark cu diferite configurații de fișiere
- prelucrarea și rulare benchmark
- încercare de optimizare
- analiza datelor obținute

2. Buzuriu Roxana

- studiu lucrări științifice
- împărțirea pe familii
- structurare referat
- introducere
- descrierea problemei
- instalare pe windows

3. Constantinescu Mihai

- studiu cod - analiza structurii generale a codului, structurilor de date folosite, funcțiilor principale folosite în program
- realizare diagrame

4. Felea Irina

- studiu cod - analiza parametrilor utilizați pentru rularea fișierelor
- împărțirea pe familii
- studiu familii
- studiu al lucrărilor științifice în care sunt descrise problemele abordate în familia aleasă
- rulare benchmark
- generare licență